

telemetryDisplay

Application web de visualisation de télémétrie pour circuit, avec import de fichiers `.mat`, dashboard de graphes, track map et création de math channels.

Structure du projet

- `backend/` : API FastAPI et logique d'import des données
- `frontend/` : interface React + Vite
- `data/` : jeux de données d'exemple et cartes de piste
- `docs/` : conventions de format, dont le format MAT

Prérequis

- Python 3.10+ pour le backend
- Node.js 18+ pour le frontend
- Un navigateur moderne

Pour installer Node.js, rendez vous sur <https://nodejs.org/en/download/current> et téléchargez le Windows Installer. Suivez les étapes d'installation.

Pour vérifier que les prérequis sont validés vous pouvez taper les commandes suivantes dans un terminal bash:

- `python --version` et vous verrez la version de votre Python apparaître.
- `npm --version` et vous verrez également la version de votre Node.js apparaître.

Installation

Backend

1. Ouvrir un terminal dans `telemetryDisplay`.
2. Créer l'environnement virtuel si besoin: `python -m venv .venv`
3. Activer l'environnement virtuel.
4. Installer les dépendances: `pip install -r requirements.txt`

Frontend

1. Ouvrir un terminal dans `frontend/`.
2. Installer les dépendances: `npm install`

Lanceur global

1. Retourner dans `telemetryDisplay`.
2. Installer les dépendances: `npm install`.

Démarrage

Pour une utilisation quotidienne, préférez le démarrage complet depuis la racine.

Lancer le backend

1. Aller dans `backend/`.
2. Activer `.venv`.
3. Lancer l'API: `uvicorn app.main:app --reload --port 8001`

L'API est disponible sur `http://localhost:8001`.

Lancer le frontend

1. Aller dans `frontend/`.
2. Lancer l'interface: `npm run dev`

L'application est disponible sur `http://localhost:5173` et s'attend à trouver l'API sur `http://localhost:8001`.

Démarrage simultané du frontend et du backend

1. Aller dans `telemetryDisplay`.
2. Lancer l'application au complet avec la commande: `npm run dev`

Vous verrez alors se lancer une fenêtre Node où les informations du frontend seront en cyan et celles du backend en vert.

Démarrage complet depuis la racine

Depuis la racine du dépôt, vous pouvez utiliser le script de dev configuré pour lancer frontend et backend ensemble. Ce mode suppose que les dépendances sont déjà installées.

Afin de simplifier l'utilisation, vous pouvez créer un raccourci de ce script sur le bureau.

Utilisation basique

1. Importer un fichier `.mat` depuis le panneau Data Hub.
2. Ou importer un fichier depuis un chemin local si le backend y a accès.
3. Consulter les signaux disponibles dans la liste de gauche.
4. Ajouter des math channels depuis le même panneau pour créer des signaux dérivés.
5. Ajouter des cartes et modifier les gains/offsets en fonctions de vos besoins.
6. Ouvrir le dashboard pour afficher les graphes et la track map.

Fonctions utiles de l'interface:

- Le panneau Data Hub permet d'importer, filtrer et ajouter des math channels.
- Les math channels sont conservés au rechargement de la page.
- Le mode `Graph Only` masque les panneaux latéraux pour se concentrer sur les graphes.
- Le panneau Inspecteur sert à modifier widgets, tailles et positions.
- L'aide des raccourcis clavier est accessible depuis le bouton `?` en haut à droite.
- L'onglet `Trajectoire` permet de visualiser la trajectoire de la voiture dans le plan quand les signaux `xCar`, `yCar`, `xTrack` et `yTrack` sont présents.
- L'onglet `Rejeu Cartos` permet de synthétiser des canaux avec des LUT qui prennent en entrées des canaux existants.

- Les canaux peuvent être visualisés de différentes façons: que les valeurs positives, que les valeurs négatives les valeurs correspondants à des phases de freins (**MBrakeR** != 0)

Format des fichiers MAT

Le format attendu est décrit dans [docs/MAT_FORMAT.md](#).

En résumé:

- **sLap** est obligatoire et doit être monotone croissante.
- Chaque signal doit avoir la même longueur que **sLap**.
- **distance_step_m** est recommandé pour décrire l'échantillonnage spatial.
- Les **NaN** dans le signal sont acceptés, s'ils sont en fin de signal et uniquement en fin de signal on trim les données du dataset (explosion d'une simulation)

Données d'exemple

- **data/losail.mat** : dataset de démonstration pour Losail
- **data/losail_track.csv** : tracé de piste associé
- Les scripts de génération se trouvent dans **backend/scripts/**

Utilisation avec MATLAB / Simulink

Cette application vise à offrir un outil pratique pour analyser des données de télémétrie issues d'une simulation Simulink. Pour générer les données compatibles avec l'application, procédez ainsi:

1. Copier le fichier **exportDataMATLAB.m** dans le dossier de votre projet Simulink.
2. Le configurer comme callback **StopFcn** de la simulation.
3. Ajouter des blocs **To Workspace** sur les signaux que vous voulez analyser dans l'application.

Prise en compte des mise à jour du code

Lors du développement de l'application, des mises à jour régulières seront disponibles avec des résolutions de bugs ou des nouvelles fonctionnalités, voici la démarche pour les récupérer:

1. Aller dans le dossier où est le code source de l'application (nommé **telemetryDisplay**) et assurez vous qu'aucune instance de l'application n'est ouverte.
2. Clic droit dans la fenêtre et faites **Open Git Bash here**.
3. Assurez vous d'avoir une connexion internet.
4. Tapez dans le terminal ouvert les commandes suivantes:
5. **git fetch**
6. **git checkout {nom de la branche}** avec le nom de la branche donné dans le message indiquant la mise à jour (étape souvent optionnelle)
7. **git pull**
8. Relancez l'application avec le script **Telemetry Display.sh**

Si besoin de mettre à jour les bibliothèques Python:

1. **source .venv/Scripts/activate**
2. **python -m pip install -r requirements.txt**

3. `deactivate`

Si besoin de mettre à jours les dépendances Node:

1. `npm install`
2. `cd frontend`
3. `npm install`
4. `cd ..`